Part 1

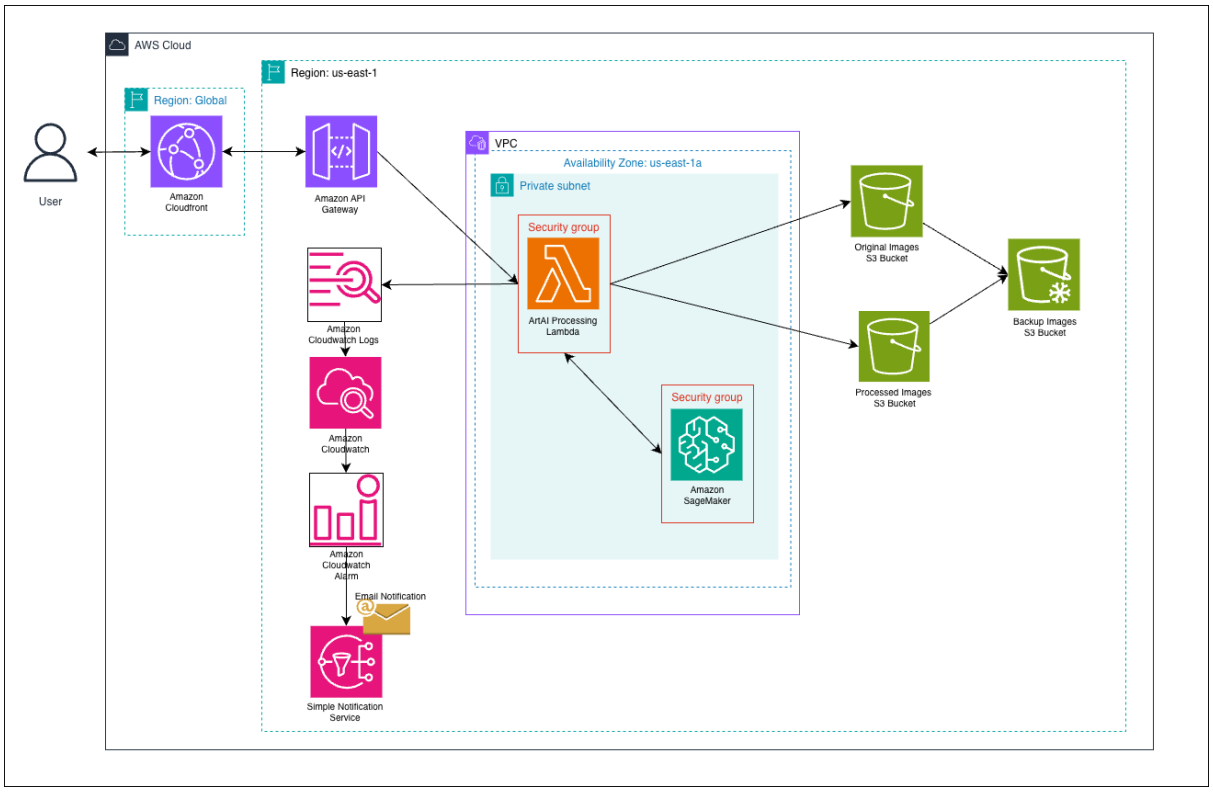


Figure1. ArtAI webservice AWS architecture.

This AWS architecture implements a synchronous service where users can upload images through API POST requests and immediately receive AI model processed image results.

To achieve global performance and availability, we choose Amazon CloudFront service. It is a global Content Delivery Network (CDN) that can provide high availability and low latency by using edge locations across the world. CloudFront caches static resources and routes API requests through Amazon API Gateway to the region where actual server is running. For example, when a user uploads the image, if the processed image already exists in the edge location, the cashed result will be returned from the nearest edge location, allowing low latency. If there's no cached asset, the request is sent to the original server, and the result is returned by the API Gateway to CloudFront to the user.

To achieve a serverless infrastructure, we choose AWS Lamba and API Gateway[1] combination over EC2 instances. This structure allows automatic scaling based on demand only. Moreover, we only choose one Lambda to operate the overall main process: uploading images to "Original Images S3 Bucket", processing original images using AI model in SageMaker, saving processed images to "Processed Images S3 Bucket", and logging CloudWatch Logs. This simplifies the architecture and reduces inter-service latency, such as asynchronous cooperation using SQS. In terms of API Gateway, besides providing API to the users, it also can seal the backend which allow Lambda to run in a

[1] Amazon Web Services, "What is Amazon API Gateway?"

protected private network[2].

Amazon SageMaker is chosen for AI model deployment since it produces inference online endpoints with auto-scaling capabilities[3]. Using with Lambda and API Gateway services, we can construct a serverless ML architecture. API Gateway passes the images and parameters to Lamda function and Lambda function parses them to a data structure that can be sent to SageMaker model endpoint. The model then returns the response to the Lambda, and the Lambda sends the processed image download URL back to API Gateway[4]. Lastly, API Gateway will give respond to the client through CloudFront as mentioned in the first paragraph. Moreover, both Lambda and SageMaker should be deployed within the same private subnet in a VPC which blocks the public internet access. Communication between Lambda and SageMaker uses Security Group rules. Lambda should allow outbound traffic to SageMaker's Security Group on port 443 that allows HTTPS only. SageMaker Security Group is set to allow only inbound traffic coming from the Lambda's Security Group. These settings will ensure the security of the AI model service.

For S3 bucket, we need original image uploading bucket and processed saving image bucket separately for better management. The Lambda function will store the image to "Original Images S3 Bucket" and when image is done processing it will be stored in "Processed Images S3 Bucket" by the Lambda. The processed image URL will be returned to the user with S3 bucket object's pre-signed URL that allows temporary access to the processed object only without permission to access S3[5]. In addition, since user can upload the specific image of different versions we can enable S3 Versioning that allows retaining multiple versions of objects in both S3 buckets. With S3 Versioning, the service can preserve, retrieve, and restore all versions of objects stored in the S3 bucket[6]. Thus, a new version will be uploaded without overwriting previous objects. Lastly, an S3 Lifecycle Policy allow automatic transitions older versions to Amazon S3 Glacier after 30days[7]. This allows reducing costs and maintain long-term backup accessibility.

Monitoring logs should be generated by Lambda function immediately upon receiving a request before any image process begins. Which means, it can ensure all counting of requests regardless of the failure in the later process for accurate traffic monitoring. CloudWatch Alarm Logs are created by Lambda and CloudWatch Alarm will monitor the metric of the requests with a 24-hour period. When the sum of requests exceeds 10,000 requests, the alarm will trigger Amazon SNS that sends an email notification to administrator.

With these services and architecture components, we can achieve secured, high performing, and cost-efficient AI image processing application.

---

[2] Olsen, "Call an Amazon SageMaker model endpoint using Amazon API Gateway and AWS Lambda."

[3] Gryczka. "Image background removal using Amazon SageMaker semantic segmentation."

[4] Olsen, "Call an Amazon SageMaker model endpoint using Amazon API Gateway and AWS Lambda."

[5] AWS Boto3 1.42.4. documentation, "Boto3 documentation."

[6] Amazon Web Services, "What is S3?"

[7] Amazon Web Services, "Transitioning objects using Amazon S3 Lifecycle."

Part 2

1. Application Architecture (Part2.A)

The WordFreq application is a distributed file processing system that calculates word frequencies in text files. This service's cloud architecture follows an event-driven and queue-based processing pattern.

1.1 Overall Processing Flow

This service working flow can be explained in big three parts as defined in section 1.1.1 – 1.1.3.

1.1.1 File Uploading Trigger Process

- Users copy uploaded .txt files from S3 Uploading Bucket to the S3 Processing Bucket.
- S3 Processing Bucket automatically generates an event notification when a new file is uploaded.
- The event notification is sent as a message to the SQS Jobs Queue.

1.1.2. Job Queue Management

- Each message stored in SQS Jobs Queue contains metadata about the uploaded file (bucket name, file key).
- These messages are read by the WordFreq application in the EC2 instances by a consistent polling method.
- Messages remain in the queue until successfully processed by an EC2 worker.

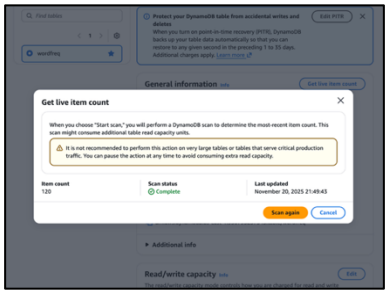
1.1.3. Worker Processing

The EC2 worker instance continuously polls the SQS Jobs Queue.

If there is one or more messages:

- Worker downloads the specified file from the S3 Processing Bucket.
- Performs word frequency calculation.
- Stores results in DynamoDB table in ("Filename": map of word counts) structure and at the same time sends completion notification to SQS Results Queue.
- Deletes the original message from the Jobs Queue to prevent reprocessing.

1.2 Testing Results

(a) 

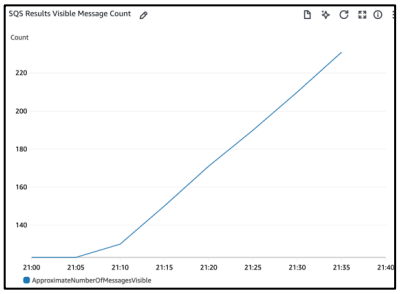
(b) 

Figure.1.1. a) 120 results saved in Dynamo DB. b) Approximate number of messages visible in SQS Results.

After finishing implementing the basic WordFreq application service architecture, we can test by using the CLI command to copy object files from the S3 Uploading Bucket to the S3 Processing Bucket (Fig.1.1.a).

1.3 Analysis

Initial Log	Last log	Total Time
Nov 20 21:15:06 ip-172-31-28-71 wordfreqservice[760]: Processing message de1f80b8-021b-4fe0-b8b1-5088149b6d0f	**Nov 20 21:42:26** ip-172-31-28-71 wordfreqservice[760]: Deleted message, a05d4070-f515-4b00-9370-351393886c6b	**00:27:20**

Table.1.1. CLI time logs of initial process time and last process time log.

Since we only have one instance processing the uploaded files, the processing time took 27 minutes.

2. Auto-Scaling Implementation (Part2.B)

The auto-scaling implementation uses SQS depth as the primary performance metric rather than CPU utilisation. It is because our service operation is dependent on the SQS's visible message count, and due to 10-20 seconds of forced wait time, it keeps the CPU usage consistently low. Thus, queue depth can accurately reflect the status of additional processing capacity.

2.1 CloudWatch Alarms and Scaling Policies

	Scale Out Alarm (wordfreq-scale-out-alarm)	Scale In Alarm (wordfreq-scale-in-alar)
Executed Scaling Policy	• Resource: wordfreq-auto-scaling-group • Policy Type: Step Scaling • Policy: wordfreq-scale-out-policy	• Resource: wordfreq-auto-scaling-group • Policy Type: Step Scaling • Policy: wordfreq-scale-in-policy
Policy Execution Metrics	• SQS ApproximateNumberOfMessagesVisible • (QueueName = wordfreq-jobs)	• SQS ApproximateNumberOfMessagesVisible • (QueueName = wordfreq-jobs)
Conditions and Scaling Threshold	• Statistic: Average • Period: 1 minute • Threshold Type: Static • Condition: messages in SQS Jobs >= 10 • Evaluation Periods: 1 out of 1	• Statistic: Average • Period: 1 minute • Threshold Type: Static • Condition: messages in SQS Jobs < 5 • Evaluation Periods: 1 out of 1
Actions	• Alarm State Trigger: In alarm • Action Type: Auto Scaling • Scaling Adjustment: Add 1 capacity unit • Adjustment Type: ChangeInCapacity • **Instance Warmup: 120 seconds (satisfies 2-minute interval requirement)**	• Alarm State Trigger: In alarm • Action Type: Auto Scaling • Scaling Adjustment: Remove 1 capacity unit • Adjustment Type: ChangeInCapacity

Table.2.1. Cloud alarms and scaling policies.

For CloudWatch Alarm configuration, we make two types of alarm: scale-out alarm and scale-in alarm. We use two predefined scaling policies as triggers. When the number of SQS messages reaches or exceeds 10, scale-out alarm status changes to “in alarm” and after 2-minute of warmup time, a new EC2 instance is launched. When the number of SQS messages falls below 5, the scale-in alarm changes to “in alarm” status and ASG start termination to the most recently launched instance.

2.2 Testing Results – Baseline Load Test

2.2.1 S3 Processing Bucket - File Distribution

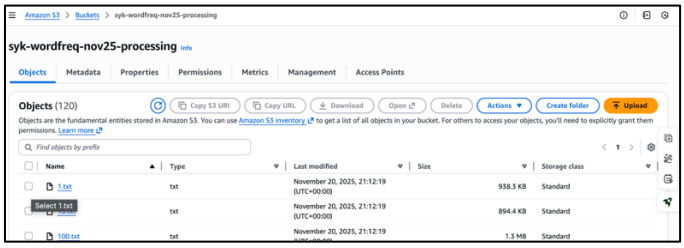


Figure.2.1. File objects successfully uploaded on S3 Processing Bucket.

Using the CLI command, we successfully copied 120 files from the S3 uploading bucket to the S3 processing bucket.

2.2.2 Auto Scaling Group’s Scaling Activity History

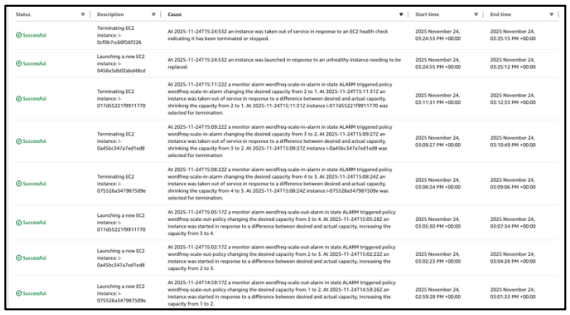


Figure2.2. Auto Scaling Group Activity History for the Baseline test.

Fig2.2. illustrates the EC2 instance lifecycle. As shown in the "Cause" column, all scaling activities were triggered by CloudWatch alarms.

2.2.3 EC2 Instance Launch and Termination Status

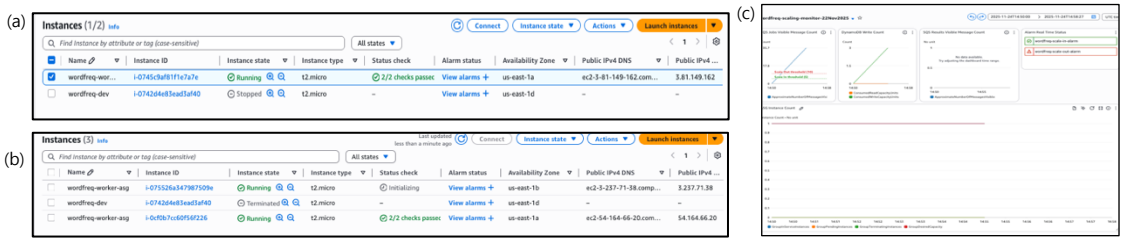


Figure2.3. a) Initial EC2 Instance Launched by Auto Scaling Group. b) Scale-out alarm triggered when the 1-minute average count of visible messages in the SQS exceeded the threshold of 10. c) CloudWatch dashboard showing scale-out alarm status in “in alarm” status.

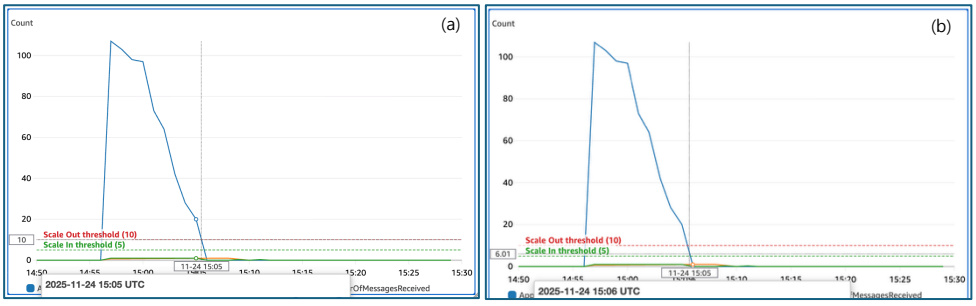


Figure.2.4. SQS Jobs Queue approximate visible message count.

The average visible message number falls to 10 after 13:05 UTC which will stop launching new EC2 instances (Fig.2.4.a). Around 15:06 UTC, the average number of visible messages in the SQS jobs queue falls to 5 (Fig.2.4.b), which will initiate termination of existing EC2 instances until desired minimum capacity is met.



Figure.2.5. a) Scaled-out EC2 instances. b) Shutting down EC2 instances. c) Scaled-in EC2 instances.

Fig.2.5 shows the launched, shutting down and terminated instances during scale out and scale in.

2.2.4 DynamoDB Results - Processed Data Output

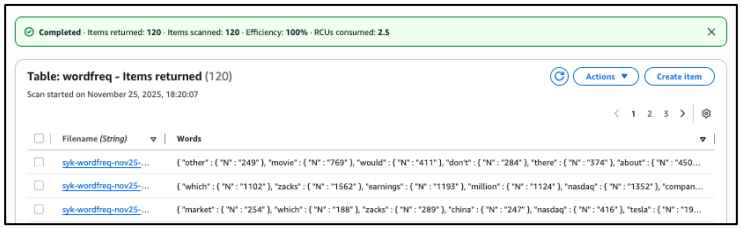


Figure.2.6. DynamoDB with 120 results created.

Upon completion of file processing in the WordFreq application, it also creates a data row in DynamoDB with two fields: "Filename" (string type) and "Words" (dictionary type with word keys and frequency information values).

2.2.5 Analysis of Baseline Test

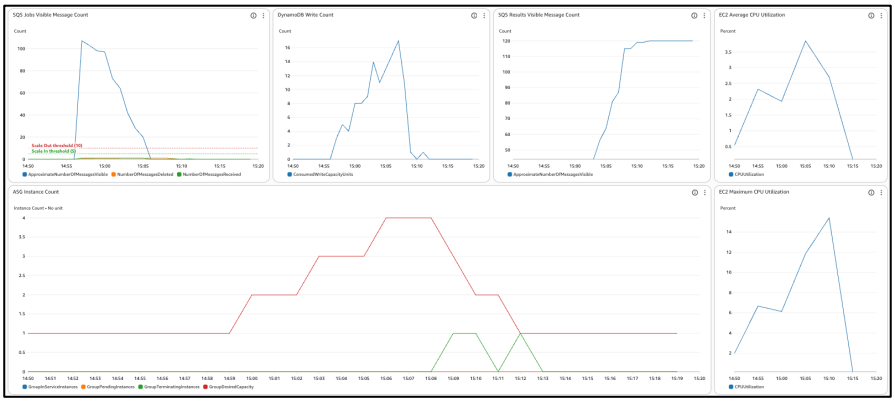


Figure 2.7. CloudWatch dashboard of key AWS service metrics.

The baseline test (14:57–15:12 UTC, total 15 minutes, throughput of 8 files/min) processed 120 files using t2.micro (1 vCPU), reducing processing time by 44.4% versus single EC2 instance execution. However, there are key observations indicating bottlenecks of this setting.

ASG scaled out to only 4 instances in maximum, while the max instance number was set to 5. In addition, the highest 1-minute average was 3.85% and the maximum CPU utilisation peaked only 15.41% during the process duration. With 2-minute of launch interval, the SQS Jobs reached its peak at 14:57 and remained elevated until 15:05, but instance count reached maximum only at 15:05. This result implies insufficient parallelisation due to delayed scale-out.

Thus, to find optimal settings, we can test the reduced warmup time for faster provision and lower the scale-out threshold value for earlier capacity addition. Lastly, while single vCPU of

t2.micro instance is not in the bottleneck, we still can confirm whether the multi vCPU can contribute to higher throughput or I/O bound persists.

3. Load Testing (Part2.C)

3.1 CPU Core Count Impact Evaluation

3.1.1 EC2 Type Selection

Instance type	vCPUs	Memory (GiB)	Socket	Cores per Socket	Threads per Core
t2.micro	1	1	1	1	1
t2.medium	2	4	1	2	1
t2.xlarge	4	16	1	4	1
t2.2xlarge	8	32	1	8	1

Table3.1. Overview of the number of vCPUs, memory size, and CPU topology (sockets, cores per socket, threads per core) for selected AWS EC2 T2 instance types.

In this experiment, we will test solely the impact of CPU type on I/O-bound workloads. We maintain settings same as the baseline load testing except for the EC2 types. Four instance types were selected with varying vCPU (Table.3.1).

3.1.2 Multi-vCPU Performance Comparison

T2.2xlarge (8 vCPU) was excluded from final result due to initialisation failures. This simultaneous instance creation caused memory over usage during instance startup, resulting in "Client. UserInitiatedShutdown".

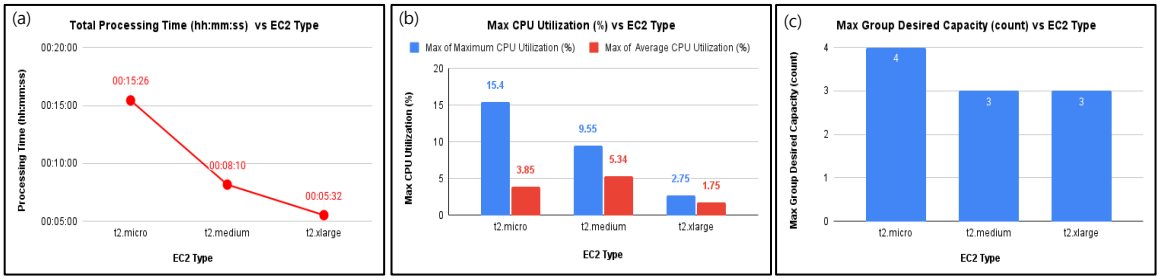


Figure.3.1. Comparative analysis of EC2 instance types (t2.micro, t2.medium, t2.xlarge): a) Total processing time. b) Max of CPU utilisation. c) Max Group Desired Capacity metric.

The t2.micro (1 vCPU) shows the longest processing time (15 minutes 26 seconds) and the with ASG scaling to 4 instances. Also, both CPU utilisation is the highest (maximum max: 15.4%, average max: 3.85).

t2.medium (2vCPUs) demonstrates a 47% improvement in processing time (total 8 minutes 10 seconds) thanks to two running concurrent workers. It achieved 9.55% maximum CPU utilisation but 5.34% average utilisation which is the highest among all tested types. The ASG scaled to 3 instances, which shows how the concurrent process can deal with tasks faster than a single core process.

The t2.xlarge instance (4 vCPUs) performs the fastest processing time at 5 minutes 32 seconds which shows 32% improvement over t2.medium and 65% time reduction of t2.micro. This shows 4 cores accelerates throughput of the process. The CPU engagement remains at 2.75% maximum and 1.75% average, also requiring 3 desired capacities, indicating over-provisioning.

3.1.3 Analysis: I/O-Bound Confirmation

Section 3.1.2 demonstrates two main observations. First, multi-core instances significantly reduced processing time through parallelization. Second, auto-scaling lagged behind peak message counts. These observations confirm that increasing the vCPU number improves performance of the overall process thanks to concurrent workers handling multiple tasks simultaneously.

3.2 Auto Scaling Configuration Optimization

3.2.1 Warmup Time Tuning

To allow maximum number of EC2 instances, we set the ASG's max desired capacity to 8, and use the same baseline test settings. In this section, we test warmup time interval of 60, 30, and 10 seconds.

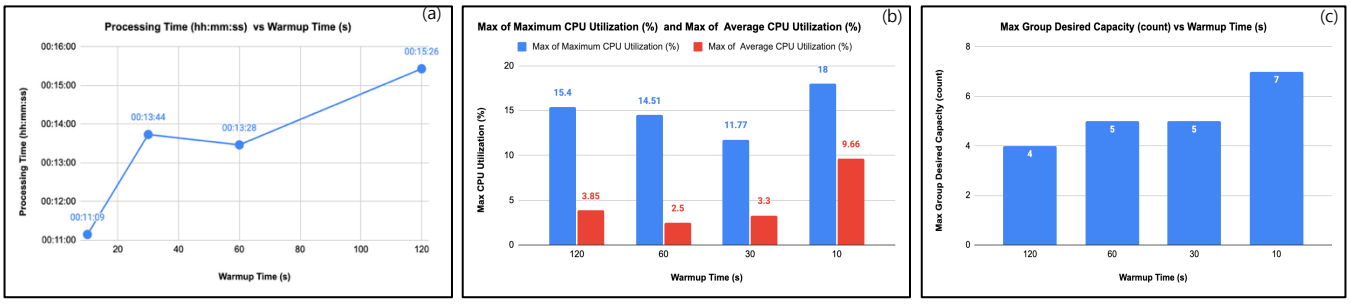


Figure.3.2. Comparative analysis of warm-up time interval: a) Total processing time. b) Max of CPU utilisation. c) Max Group Desired Capacity metric.

However comaprative results demonstrate non-linear behavior where shorter warmup times do not uniformly improve performance.

Reducing warmup to 60 seconds improved processing time to 13 minutes 28 seconds (13% faster than baseline) and enabled scaling to 5 instances. However, average CPU utilisation dropped to 2.5% which indicates that instances were spending excessive time idle.

The 30-second warmup produced worse results than 60 seconds, with processing time increasing to 13 minutes 44 seconds despite scaling to 5 instances. Average CPU utilisation remained 3.3% while maxmium peaked 11.77% indicatin uneven worklaod distribution.

The 10-second warmup achieved the fastest processing time at 11 minutes 9 seconds (28% improvement over baseline) and 17% faster than the 60-second warmup time configuration. The ASG scaled to 7 instances, the highest among all tests. Average CPU utilization reached 9.66%. This implies consistent workload distribution and active CPU usage rather then staying in idel status. Maximum CPU utilization of 18% remained within reasonable bounds.

In conclusion, the 10-second warmup time is recommended which enables the fastest processing (11 mintues 9 seconds), maximum parallelization (7 instances), and highest resource efficiency (9.66% average CPU utilization).

3.2.2 Scale-Out Threshold Tuning

To allow maximum number of EC2 instances, we set the ASG's max desired capacity to 8, and use the same baseline test settings. Then we choose scale-out threshold 10, 15, 20 messages to compare how the workload changes.

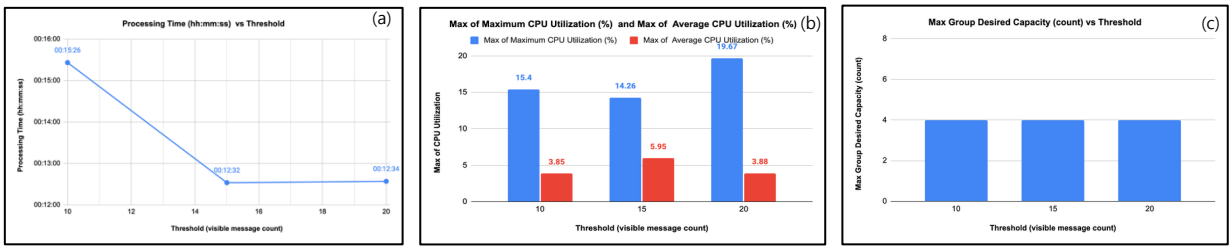


Figure.3.3. Comparative analysis of scale-out threshold: a) Total processing time. b) Max of CPU utilisation. c) Max Group Desired Capacity metric.

However, comparative analysis reveals that lower thresholds do not show faster overall processign time. Scale-out threshold set to 20 and 15 was apporaximately 3 minutes faster in processing compared to the baseline model with 10 messages as scale-out threshold. This is due to the number of SQS messages maintaining over 10 messages for siginificant time, which cause the ASG to continuosly re-evluate scaling decisions, and result in instances spending more time in installisation rather than actively processing.

The threshold of 15 messages is chosen to be optimal since it showed successfule reduction in evaluation while maintaining responsive capacity expansion. Max of average CPU utilisation increased from 3.85% (baseline) to 5.95%, while the maximum CPU utilisation decreased from 15.4% (baseline) to 14.26%. indicating more effective instance engagement. In terms of the threshold of 20 messages, it produced nearly identical processing time (00:12:34) but with a higher maximum CPU spike (19.67%), implying delayed scaling caused temporary overload in instances.

3.2.3 Optimised Auto Scaling Configuration Testing Results and Analysis

(a)

```
Dec 08 03:06:11 ip-172-31-37-164 wordfreqservice[906]: Job Message queue
Dec 08 03:06:11 ip-172-31-37-164 wordfreqservice[906]: Worker 1 starting
Dec 08 03:06:11 ip-172-31-37-164 wordfreqservice[906]: Worker 0 starting
Dec 08 03:06:11 ip-172-31-37-164 wordfreqservice[906]: Worker 2 starting
Dec 08 03:06:11 ip-172-31-37-164 wordfreqservice[906]: Worker 3 starting
```

(b)

Configuration	Processing Time (hh:mm:ss)	Max of Maximum CPU (%)	Max of Average CPU (%)	Max Desired Capacity Instances (Count)
Baseline	00:15:26	15.4	3.85	4
Optimized	00:06:29	3.83	1.71	5
Improvement	00:08:57	11.57	2.14	+1
Improvement (%)	58.0%	75.1%	55.6%	+25%

(c)

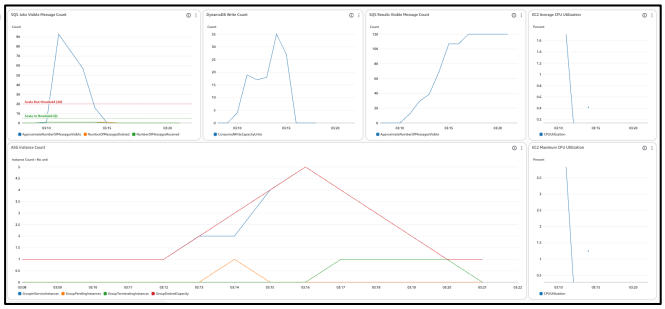


Figure.3.4. a) Worker 0-3 initiating logs in t2.xlarge EC2 instance type. b) Performance comparison between baseline and optimised Auto Scaling configurations (Improvement percentage calculated as: (Baseline - Optimized) / Baseline × 100 %). c) Load testing results with CloudWatch dashboard metrics for optimised tuning.

In this section we test final combination with optimised settings: EC2 instance type=t2.xlarge, warmup time=10-second, threshold=15-message. Also, we allow the max instance number of EC2 to be 8 to allow sufficient capacity.

As the t2.xlarge has 4 vCPU, we can confirm Worker 0-3 are initiated in the CLI processing

logs (Fig.3.4.a). The overall performance metrics showed improvement in the optimised Auto Scaling configuration compared to the baseline (Fig.3.4.b). The processing time decreased to 58%, resulting only 6 minutes 29 seconds total. The max CPU utilization values also decreased for both maximum and average CPU utilization to 75.1% and 55.6% respectively. Especially, maximum CPU utilization of 3.83% shows there were almost no overloading in CPU usage, confirming stability of processing. Moreover, the max desired capacity increased from 4 to 5 confirming shorter warmup time allowed ASG to fully utilise instance capacity.

Dashboard observations (Fig.3.4.c) show visible messages in SQS Jobs peaked at 03:11, and EC2 instances met the max requirement at 03:16 showing only delayed interval of 5 minutes. This result reflects 3 minutes improvement compared to the baseline configuration.

In conclusion, the optimised Auto Scaling configuration achieved significant performance improvements over the baseline. The optimized combination of 10-second warmup and 15-message threshold enabled the ASG to scale faster with considerable stability with t2.xlarge instances providing sufficient parallelisation increasing throughput rate.

4. WordFreq Architecture Optimisation (Part2.D)

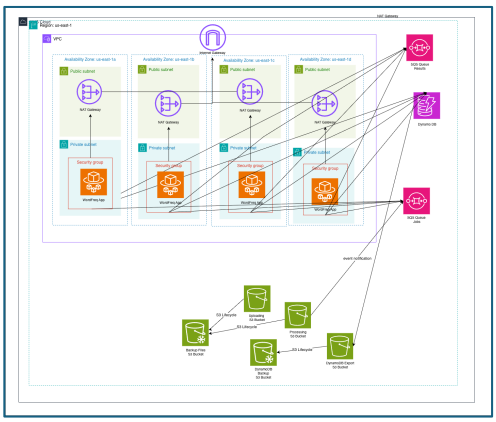


Figure.4. Optimised architecture of WordFreq application service.

For the occasional-use Go application that requires non-immediate and potentially long-running processing, Amazon ECS with AWS Fargate launch type was selected. Fargate automatically controls all server management by allowing AWS to handle provisioning, patching, and scaling infrastructure. It bills only for the vCPU and memory consumed during task execution, resulting in zero cost when idle. While Lambda is another serverless option suitable for event-driven tasks, it has 15-minute limited time of processing. However, Fargate supports unlimited execution duration providing stability for the service. Therefore, Fargate represents the most cost-effective, stable, and flexible solution for the occasional task.[8].

To increase resilience and availability of the application against component failure, we can set four AZs in the same region. Each AZ contains both private and public subnet to ensure redundancy. This when one AZ fails, the traffic is rerouted to remaining healthy AZs allowing

[8] Amazon Web Services, “Architect for AWS Fargate for Amazon ECS.”

continuous operation. The NAT Gateways are also deployed in each public subnet to ensure private subnets to access AWS services and the internet while keeping security.

Processing S3 Bucket and Uploading S3 Bucket data can be stored in Backup Files S3 Glacier Bucket for long-term storage with S3 Lifecycle. For data in DynamoDB, AWS Backup is recommended for long-term backups since general backup function for DynamoDB only lasts for 35days[9][10]. We thus choose to use DynamoDB "Export to S3" exporting full table snapshots to an S3 bucket. Then a S3 Lifecycle policy will be applied, and valuable information will be stored in S3 Glacier[11].

This architecture prevents unauthorised access through multiple security layers. First, Fargate are deployed in private subnets preventing direct public internet access. These private subnets route outbound traffic by NAT Gateways allowing Fargate to access external resources while preventing inbound connections form the internet. Second, Security Groups should be configured with the principle of leas privilege thus allowing only outbound traffic to specific AWS services. Third, IAM roles attached to Fargate will only grant minimum permissions for DynamoDB and SQS queues. Finally, VPC endpoints can be configured for AWS services that will ensure traffic between Fargate tasks and services remains within AWS network.

5. Apache Hadoop and Apache Spark (Part2.E)

Hadoop consists of HDFS (Hadoop Distributed File System) for distributed storage and MapReduce for parallel data processing. The MapReduce computes in two phases: the Map phase, where words are counted in individual text file chunks parallelly processed across nodes, and the Reduce phase, where word counts are aggregated across all chunks, producing final work frequencies map result[12].

Hadoop offers four advantages over the current WordFreq architecture. First, it provides massive scalability, allowing the system to scale from gigabytes to petabytes of data across thousands of nodes. Second, HDFS splits large files into blocks and replicates them across multiple nodes enabling fault tolerance. If a node fails during processing, healthy node is reassigned to the task with replicated data blocks, whereas the WordFreq system requires manual SQS message retry mechanisms. Third, Hadoop schedules computation on nodes where data physically exists, which minimise network transfer overhead. In contrast, WrodFreq app requires each EC2 worker to download files from S3 for every job, creating significant I/O bottlenecks. Lastly, Hadoop's automatic load balancing distributes work across available nodes based on their current capacity, while the WrodFreq app relies on SQS message distribution which cannot engage varying instance capacity.

Unlike Hadoop's disk-based processing model, Spark utilises in-memory computation through Resilient Distributed Datasets (RDDs), which are immutable distributed collections of objects

---

[9] Amzaon Web Services, "Backup and restore for DynamoDB."

[10] Amazon Web Services, "What is AWS Backup?"

[11] Amazon Web Services, "Using DynamoDB with Amazon S3 to export and import table data."

[12] The ASF, "Hadoop-AWS module: Integration with Amazon Web Services."

cached in RAM across cluster nodes[13]. This eliminates the intermediate disk I/O required in MapReduce stages, and keeps data in memory throughout multi-stage processing. Spark would load text files into RDDs, apply transformations. The flatMap splits words, maps to create key-value pairs, and reduce by key to count frequency of the words. This is done entirely in memory, and write only final results to storage.

Spark can deliver performance improvements over the current WordFreq implementation in two ways. First, in-memory processing provides faster execution compared to disk-based systems like the current EC2-based architecture. Lastly, Spark's dynamic resource allocation automatically scales number of the executors based on workload demand. It releases resources when idle, optimising cost-efficiency for variable workloads[14].

6. Reference

AWS Boto3 1.42.4. Documentation. 2025. Boto3 documentation. Accessed 12 08, 2025. https://boto3.amazonaws.com/v1/documentation/api/latest/index.html.

Amazon Web Services. 2025. Architect for AWS Fargate for Amazon ECS . Accessed 12 08, 2025. https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html.

Amazon Web Services. 2025. Configure Spark . Accessed 12 08, 2025. https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-spark-configure.html#spark-defaults.

Amzaon Web Services.2025.Backup and restore for DynamoDB. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Backup-and-Restore.html.

Amazon Web Services. 2025. What is AWS Backup? . Accessed 12 08, 2025. https://docs.aws.amazon.com/aws-backup/latest/devguide/whatisbackup.html.

Amazon Web Services. 2025. Transitioning objects using Amazon S3 Lifecycle. Accessed 12 08, 2025. https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-transition-general-considerations.html.

Amazon Web Services. 2025. What is Amazon S3? Accessed 12 08, 2025. https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html.

Amazon Wev Services. 2025. What is Apache Spark? . Accessed 12 08, 2025. https://aws.amazon.com/what-is/apache-spark/.

Patrick Gryczka, Ajit Puthiyavettle, and Kenny Guzman. 2022. Image background removal using Amazon SageMaker semantic segmentation. 7. Accessed 12 08, 2025. https://aws.amazon.com/ko/blogs/architecture/image-background-removal-using-amazon-sagemaker-semantic-segmentation/.

[13] Amazon Wev Services, “What is Apache Spark?”

[14] Amazon Web Services, “Configure Spark .”

Rumi Olsen, Achintya Singh, and Rohit Raj. 2025. Call an Amazon SageMaker model endpoint using Amazon API Gateway and AWS Lambda. 03. Accessed 12 08, 2025. https://aws.amazon.com/ko/blogs/machine-learning/call-an-amazon-sagemaker-model-endpoint-using-amazon-api-gateway-and-aws-lambda/.

Services, Amazon Web. 2025. What is Amazon API Gateway? Accessed 12 08, 2025. https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html.

The ASF. 2025. Hadoop-AWS module: Integration with Amazon Web Services. 8 20. Accessed 12 08, 2025. https://hadoop.apache.org/docs/stable/hadoop-aws/tools/hadoop-aws/index.html.

Zamborsky, Ted. 2023. Backup strategies for Amazon DynamoDB . 05 19. Accessed 12 08, 2025. https://aws.amazon.com/ko/blogs/database/backup-strategies-for-amazon-dynamodb/.